

HS Factor Hub v2 — Blueprint

Documento de referencia completo antes de iniciar construcción. Captura todas las decisiones de arquitectura, schema, endpoints y páginas acordadas.

Contexto

El Hub v1 fue generado con Lovable sobre TanStack Start + Supabase Auth + Cloudflare Workers. Funciona pero tiene deuda técnica significativa: credenciales hardcodeadas, tokens de sesión en URL, migraciones incompletas, y toda la lógica acoplada a Supabase. El Hub v2 migra al stack estándar del equipo con infraestructura real.

El Hub es la plataforma interna de HS Factor. Hoy tiene CRM + dashboard de herramientas. En el futuro absorberá las otras apps del ecosistema (Diagnóstico Scaling Up, Lead Magnets, generación de contenido).

La base de datos Supabase es **compartida** entre el Hub y esas otras apps. El Hub v2 vive en un schema propio (`hub`) dentro del mismo Postgres de Supabase — convive sin conflicto con las tablas existentes en `public` .

Stack

Capa	Tecnología	Notas
Frontend	Next.js 14+ App Router	TypeScript, Tailwind v4, <code>output: standalone</code>
Backend	FastAPI + Pydantic V2	Autodocs en <code>/docs</code>
Base de datos	PostgreSQL (Supabase)	Schema <code>hub</code> propio, Alembic gestiona solo ese schema
ORM	SQLAlchemy	Modelos declarativos
Migraciones	Alembic	Nunca editar la DB a mano
Auth	JWT (python-jose)	Access token + refresh token, roles en payload
Emails	Resend	Invitaciones, reset de contraseña
Archivos	Supabase Storage	FastAPI accede con <code>service_role</code> key
Secrets	Infisical	Fuente única de verdad
Deploy	Coolify	Un proyecto, dos recursos: backend + frontend
CI/CD	GitHub Actions → Coolify webhook	Build por servicio según cambios

Estructura del Repo

```
/
├── backend/
│   ├── app/
│   │   ├── routers/
│   │   │   ├── auth.py
│   │   │   ├── usuarios.py
│   │   │   ├── crm.py
│   │   │   ├── actividad.py
│   │   │   ├── archivos.py
│   │   │   └── contenidos.py
│   │   ├── models/
│   │   │   ├── user.py
│   │   │   ├── oportunidad.py
│   │   │   ├── actividad.py
│   │   │   ├── archivo.py
│   │   │   └── tema_contenido.py
│   │   ├── schemas/
│   │   │   ├── auth.py
│   │   │   ├── usuario.py
│   │   │   ├── oportunidad.py
│   │   │   ├── actividad.py
│   │   │   ├── archivo.py
│   │   │   └── contenido.py
│   │   ├── services/
│   │   │   ├── auth.py
│   │   │   ├── email.py          ← Resend
│   │   │   ├── storage.py       ← Supabase Storage
│   │   │   └── activecampaign.py
│   │   ├── core/
│   │   │   ├── config.py        ← Settings (pydantic-settings)
│   │   │   ├── database.py     ← SQLAlchemy engine + session
│   │   │   └── security.py     ← JWT, bcrypt
│   │   └── db/
│   │       └── seed.py
│   ├── alembic/
│   ├── Dockerfile
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── api-proxy/[...path]/route.ts
│   │   │   ├── (auth)/
│   │   │   │   ├── login/page.tsx
│   │   │   │   ├── reset-password/page.tsx
│   │   │   │   └── set-password/page.tsx
│   │   │   ├── (hub)/
│   │   │   │   ├── layout.tsx          ← verifica sesión
│   │   │   │   ├── dashboard/page.tsx
│   │   │   │   ├── crm/page.tsx
│   │   │   │   ├── crm/[id]/page.tsx
│   │   │   │   ├── admin/page.tsx
│   │   │   │   └── contenidos/page.tsx
│   │   │   └── components/
│   │   │       ├── crm/
│   │   │       │   ├── KanbanBoard.tsx
│   │   │       │   ├── KanbanColumn.tsx
│   │   │       │   ├── OportunidadCard.tsx
│   │   │       │   ├── EtapaTimer.tsx
│   │   │       │   ├── NuevaOportunidadModal.tsx
│   │   │       │   └── FiltrosCRM.tsx
│   │   │       ├── ui/                ← shadcn/ui
│   │   │       └── lib/
│   │   │           ├── api.ts         ← axios hacia /api-proxy
│   │   │           ├── auth.ts       ← helpers de sesión
│   │   │           └── types/
│   │   │               ├── crm.ts
│   │   │               └── user.ts
│   ├── Dockerfile
│   ├── next.config.mjs
│   └── docker-compose.yml
├── .github/
│   └── workflows/
│       └── backend.yml
```

Schema de Base de Datos — `hub.*`

Alembic gestiona exclusivamente el schema `hub`. Las tablas en `public.*` las dejan intactas (otras apps las usan).

`hub.users`

Reemplaza `auth.users` + `public.user_roles` + `public.profiles` para el Hub. Los UUIDs de usuarios existentes se preservan para mantener referencias históricas en el CRM.

Columna	Tipo	Notas
<code>id</code>	<code>uuid</code>	PK, mismo UUID que tenían en Supabase Auth
<code>email</code>	<code>text</code>	Unique, not null
<code>hashed_password</code>	<code>text</code>	bcrypt, not null
<code>role</code>	<code>enum</code>	<code>superadmin</code> , <code>coordinador</code> , <code>vendedor</code>
<code>is_active</code>	<code>bool</code>	Default true
<code>created_at</code>	<code>timestampz</code>	Default now()
<code>updated_at</code>	<code>timestampz</code>	Default now()

`hub.crm_oportunidades`

Copia de `public.crm_oportunidades`. Los campos `vendedor_id`, `created_by`, `eliminado_por` referencian `hub.users.id`.

Columna	Tipo	Notas
id	uuid	PK
nombre	text	Not null
email	text	Nullable
telefono	text	Nullable
empresa	text	Nullable
tamano	text	Nullable
ubicacion	text	Nullable
fuente	text	Nullable
etapa	enum	a_discovery , discovery_agendada , en_diagnostico , presentacion_agendada , en_cierre , ganado , descartado
vendedor_id	uuid	FK → hub.users.id , nullable
producto_recomendado	text	Nullable
observaciones	text	Nullable
created_by	uuid	FK → hub.users.id , nullable
created_at	timestampz	Default now()
updated_at	timestampz	Default now()
etapa_changed_at	timestampz	Nullable
eliminado	bool	Default false
eliminado_razon	text	Nullable
eliminado_at	timestampz	Nullable
eliminado_por	uuid	FK → hub.users.id , nullable

hub.crm_actividad

Historial de acciones sobre una oportunidad.

Columna	Tipo	Notas
id	uuid	PK
oportunidad_id	uuid	FK → hub.crm_oportunidades.id
autor_id	uuid	FK → hub.users.id , nullable
tipo	text	nota , llamada , email , etapa_cambio
nota	text	Nullable
etapa_anterior	text	Nullable
etapa_nueva	text	Nullable
created_at	timestampz	Default now()

hub.archivos_clientes

Archivos adjuntos a oportunidades. El archivo físico vive en Supabase Storage bucket `archivos-clientes`.

Columna	Tipo	Notas
id	uuid	PK
oportunidad_id	uuid	FK → hub.crm_oportunidades.id
nombre	text	Nombre original del archivo
descripcion	text	Nullable
tipo	text	diagnostico , propuesta , contrato , presentacion , reporte , otro
storage_path	text	Path en Supabase Storage
subido_por	uuid	FK → hub.users.id , nullable
tamano_bytes	int8	Nullable
created_at	timestampz	Default now()

hub.temas_contenido

Banco de temas para generación de contenido. Solo superadmin puede gestionar.

Columna	Tipo	Notas
id	int	PK autoincrement
tema	text	Not null
angulo	text	Nullable
activo	bool	Default false. Solo uno activo a la vez
orden	int	Para ordenar la lista
created_at	timestampz	Default now()
updated_at	timestampz	Default now()

Tablas de solo lectura desde `public.*`

FastAPI lee estas tablas para mostrar datos en el expediente del CRM. No las gestiona ni las migra.

Tabla	Uso en el Hub
public.discovery_calls	Expediente: datos de la discovery call del prospecto
public.submissions	Expediente: diagnóstico Scaling Up del prospecto
public.personas	Al crear oportunidad: upsert del contacto
public.empresas	Al crear oportunidad: upsert de la empresa
public.ac_sync_log	Log del sync a ActiveCampaign

Backend — Endpoints

Auth — `/api/auth`

Método	Ruta	Acceso	Descripción
POST	<code>/api/auth/login</code>	Público	Email + password → access_token + refresh_token
POST	<code>/api/auth/refresh</code>	Público	Refresh token → nuevo access_token
POST	<code>/api/auth/logout</code>	Autenticado	Invalida refresh token
POST	<code>/api/auth/forgot-password</code>	Público	Manda email con link de reset (Resend)
POST	<code>/api/auth/reset-password</code>	Público	Token del email + nueva contraseña
GET	<code>/api/auth/me</code>	Autenticado	Datos del usuario actual

Usuarios — `/api/admin/usuarios`

Solo accesible para `superadmin`.

Método	Ruta	Descripción
GET	<code>/api/admin/usuarios</code>	Lista todos los usuarios con rol y estado
POST	<code>/api/admin/usuarios/invitar</code>	Crea usuario + envía email de activación (Resend)
PATCH	<code>/api/admin/usuarios/{id}/rol</code>	Cambia el rol
PATCH	<code>/api/admin/usuarios/{id}/estado</code>	Habilita o deshabilita

CRM — `/api/crm`

Método	Ruta	Acceso	Descripción
GET	<code>/api/crm/oportunidades</code>	Todos	Lista con filtros: etapa, vendedor, fuente, tamaño, búsqueda
POST	<code>/api/crm/oportunidades</code>	Todos	Crear nueva. Si tiene email: upsert persona + sync a AC
GET	<code>/api/crm/oportunidades/{id}</code>	Todos	Detalle completo
PATCH	<code>/api/crm/oportunidades/{id}</code>	Todos	Editar campos
PATCH	<code>/api/crm/oportunidades/{id}/etapa</code>	Todos	Mover etapa. Registra actividad automáticamente
DELETE	<code>/api/crm/oportunidades/{id}</code>	Coordinador + Superadmin	Soft delete con razón
GET	<code>/api/crm/oportunidades/{id}/actividad</code>	Todos	Historial de actividad
POST	<code>/api/crm/oportunidades/{id}/actividad</code>	Todos	Agregar nota, llamada o email
GET	<code>/api/crm/oportunidades/{id}/discovery</code>	Todos	Lee <code>public.discovery_calls</code> por email
GET	<code>/api/crm/oportunidades/{id}/diagnostico</code>	Todos	Lee <code>public.submissions</code> por email
GET	<code>/api/crm/vendedores</code>	Todos	Lista usuarios asignables (coordinador + vendedor + superadmin)

Archivos — /api/crm/archivos

Método	Ruta	Acceso	Descripción
GET	/api/crm/oportunidades/{id}/archivos	Todos	Lista archivos de la oportunidad
POST	/api/crm/oportunidades/{id}/archivos	Todos	Upload a Supabase Storage + registro en DB
GET	/api/crm/archivos/{archivo_id}/url	Todos	Genera signed URL temporal (120s)
DELETE	/api/crm/archivos/{archivo_id}	Propietario + Coordinador + Superadmin	Elimina de Storage y DB

Contenidos — /api/contenidos

Solo accesible para `superadmin`.

Método	Ruta	Descripción
GET	/api/contenidos/temas	Lista todos los temas ordenados
POST	/api/contenidos/temas	Crear tema
PATCH	/api/contenidos/temas/{id}	Editar tema o ángulo
DELETE	/api/contenidos/temas/{id}	Eliminar
PATCH	/api/contenidos/temas/{id}/activar	Activa este, desactiva todos los demás

Frontend — Páginas

/login

Form email + password. Llama a `POST /api/auth/login`. Guarda tokens en cookies httpOnly. Redirige a `/dashboard` si ya hay sesión.

/reset-password

Form de email. Llama a `POST /api/auth/forgot-password`. Muestra confirmación.

/set-password

Recibe token del link del email. Form de nueva contraseña. Llama a `POST /api/auth/reset-password`.

Layout protegido (hub)/layout.tsx

Middleware Next.js verifica cookie con access token. Si no hay sesión válida → redirige a `/login`. Aplica a todas las rutas del Hub.

/dashboard

Vista del proceso comercial por etapas. Grid de columnas (Atracción, Discovery, Diagnóstico, Presentación, CRM). Herramientas activas como botones, próximamente deshabilitados. Header con email, botón Admin (si superadmin), botón cerrar sesión.

/crm

Kanban board con drag & drop (dnd-kit). 7 columnas de etapas. Filtros: búsqueda por nombre/empresa, vendedor, fuente, tamaño. Botón nueva oportunidad → modal. Tarjetas con `EtapaTimer`, indicador de días sin actualizar.

/crm/[id]

Expediente completo del prospecto. Columna izquierda: datos editables + datos de sistemas (Discovery Call, Diagnóstico) + archivos adjuntos. Columna derecha: historial de actividad + agregar nota/llamada/email. Botón eliminar (coordinador + superadmin).

/admin

Solo superadmin. Tabla de usuarios con rol, estado, fecha de registro. Acciones: cambiar rol, habilitar/deshabilitar, reenviar invitación. Botón invitar usuario → modal.

/contenidos

Solo superadmin. Tabla del banco de temas con orden, tema, ángulo, estado activo. Acciones: activar, editar, eliminar. Botón agregar tema → modal.

Secretos en Infisical

Secret	Servicio	Descripción
<code>DATABASE_URL</code>	Backend	<code>postgresql://postgres:[pass]@db.eqepylvezrdlaqjgqrff.supabase.co:5432/postgres</code>
<code>SECRET_KEY</code>	Backend	32+ chars aleatorio para firmar JWT
<code>RESEND_API_KEY</code>	Backend	Para emails de invitación y reset
<code>SUPABASE_SERVICE_KEY</code>	Backend	Para acceder a Supabase Storage
<code>SUPABASE_URL</code>	Backend	<code>https://eqepylvezrdlaqjgqrff.supabase.co</code>
<code>ENVIRONMENT</code>	Backend	<code>production</code>
<code>BACKEND_URL</code>	Frontend	URL interna del backend en Coolify

CI/CD

Dos workflows independientes. Cada uno solo se dispara si cambian archivos en su carpeta.

`backend.yml`

```
on: push to main, changes in backend/**
→ build Docker image
→ push to registry
→ webhook Coolify (redploy backend)
```

`frontend.yml`

```
on: push to main, changes in frontend/**
→ fetch secrets from Infisical
→ build Docker image
→ push to registry
→ webhook Coolify (redploy frontend)
```

En GitHub Actions solo viven `INFISICAL_CLIENT_ID` e `INFISICAL_CLIENT_SECRET`. El resto de secretos viene de Infisical en tiempo de build/run.

Migración de Datos (Fase 9)

Antes del go-live, un script único copia datos de `public.*` a `hub.*`:

```
-- Usuarios: importar con mismos UUIDs
INSERT INTO hub.users (id, email, hashed_password, role, is_active, created_at)
SELECT ur.user_id, u.email, '[REQUIERE_RESET]', ur.role, true, ur.created_at
FROM public.user_roles ur
JOIN auth.users u ON u.id = ur.user_id;

-- CRM
INSERT INTO hub.crm_oportunidades SELECT * FROM public.crm_oportunidades;
INSERT INTO hub.crm_actividad SELECT * FROM public.crm_actividad;
INSERT INTO hub.archivos_clientes SELECT
  id, oportunidad_id, nombre, descripcion, tipo, storage_path, subido_por, tamano_bytes, created_at
FROM public.archivos_clientes;
INSERT INTO hub.temas_contenido SELECT * FROM public.temas_contenido;
```

Todos los usuarios reciben email de “activa tu cuenta en el nuevo Hub” para setear su contraseña. Los archivos en Supabase Storage no se mueven — el `storage_path` es el mismo.

Fases de Construcción

```
Fase 1 – Esqueleto e infraestructura local
  backend/ skeleton + docker-compose + frontend/ Next.js base

Fase 2 – Modelos y migraciones
  SQLAlchemy models hub.* + primera migración Alembic

Fase 3 – Auth backend
  Login, refresh, logout, forgot/reset password, invite user

Fase 4 – CRM backend
  Endpoints de oportunidades, actividad, archivos, vendedores

Fase 5 – Contenidos + Admin backend
  Endpoints de temas, usuarios admin

Fase 6 – Frontend auth y layout
  Login, reset/set password, layout protegido, api proxy

Fase 7 – Frontend páginas
  Dashboard, CRM kanban, expediente, admin, contenidos

Fase 8 – CI/CD y deploy
  Dockerfiles, GitHub Actions, Coolify config

Fase 9 – Migración de datos y go-live
  Script de migración, emails a usuarios, redirección de dominio
```

Lo que NO entra en v2 (por ahora)

- Las otras apps del ecosistema (Diagnóstico Scaling Up, Lead Magnets) — viven en `public.*` hasta que se decida migrarlas
- `contenido_publicado` y `tiktok_videos` — existen en la DB pero no se construye UI para ellos en v2
- Real-time / WebSockets — el CRM se refresca manualmente igual que en v1
- Celery / Redis — el sync a ActiveCampaign es síncrono en v2, Celery se agrega si hay problemas de performance

Por qué un stack definido y no Lovable

El problema de fondo con Lovable

Lovable es una herramienta de generación de código, no una plataforma de infraestructura. Lo que genera es un punto de partida — código que corre, pero sin las decisiones de ingeniería que hacen que un sistema sea mantenible, seguro y escalable. El Hub v1 es evidencia de eso:

Seguridad comprometida desde el inicio

- Las credenciales de Supabase están hardcodeadas en el código fuente. Cualquiera con acceso al repo tiene la URL y la anon key.
- Los tokens de sesión (`access_token`, `refresh_token`) se pasan como query params en la URL al abrir herramientas externas. Quedan en el historial del browser, en los logs del servidor destino y en headers de Referer. Es una vulnerabilidad real, no teórica.

Infraestructura que no controlas

- El deploy vive en Cloudflare Workers gestionado por Lovable. No hay Dockerfile, no hay pipeline de CI/CD propio, no hay forma de hacer rollback controlado.
- Si Lovable cambia su plataforma, sube precios, o cierra — el proyecto queda atrapado.

Base de datos sin historial

- Solo hay 2 archivos de migración en el repo, pero el sistema tiene más de 15 tablas. El resto se creó manualmente desde la interfaz de Lovable. No hay forma de reproducir el schema en otro entorno, hacer rollback de un cambio de DB, ni entender la evolución del schema en el tiempo.

Bugs que no puedes depurar

- El selector de vendedores en el CRM queda vacío para vendedores y coordinadores porque `get_users_with_roles` solo autoriza superadmins — y nadie lo notó porque Lovable no tiene tests, no tiene logs centralizados, y el código está mezclado en archivos de 700 líneas.

Deuda que crece sola

- Los archivos de rutas (`crm.tsx` , `crm_.$id.tsx`) tienen ~700 líneas cada uno con fetching, lógica de negocio y UI mezclados. Cada feature nueva que Lovable agrega sobre ese código lo hace más difícil de entender y modificar.

Lo que cambia con un stack definido

Propiedad real de la infraestructura Todo corre en Coolify bajo tu control. Tienes acceso completo a logs, puedes hacer rollback, puedes escalar servicios individualmente, y el costo es predecible. Si Coolify desaparece, el mismo Docker Compose corre en cualquier VPS.

Seguridad por diseño

- Los secretos viven en Infisical, nunca en el código ni en el repo.
- La autenticación es JWT gestionado por FastAPI — tokens en cookies httpOnly, no en URLs.
- Toda llamada a la DB pasa por FastAPI con validación de roles explícita. No hay forma de que el frontend acceda a datos que no le corresponden.

Base de datos versionada Alembic mantiene el historial completo del schema. Cada cambio es un archivo de migración commiteado — reversible, auditable, y reproducible en cualquier entorno. Nunca más “no sé quién agregó esa columna ni cuándo”.

Separación de responsabilidades

- El frontend solo sabe renderizar y llamar a la API.
- El backend valida, autoriza y ejecuta la lógica de negocio.
- La DB almacena datos con un schema definido y versionado.

Cuando algo falla, sabes exactamente en qué capa buscar.

CI/CD que puedes confiar Los workflows de GitHub Actions construyen y publican imágenes Docker reproducibles. El mismo artefacto que se prueba es el que se despliega. No hay magia ni pasos manuales.

Extensibilidad real El stack está diseñado para crecer: agregar Celery para tareas async, Redis para caché, un worker para procesar archivos — todo encaja porque la arquitectura tiene capas claras. Con Lovable, cada feature nueva es un parche sobre parches.

La diferencia en una línea

Lovable genera código que funciona hoy. Un stack definido construye un sistema que puedes mantener, depurar, escalar y entender en dos años.